

Docker and Kubernetes



Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

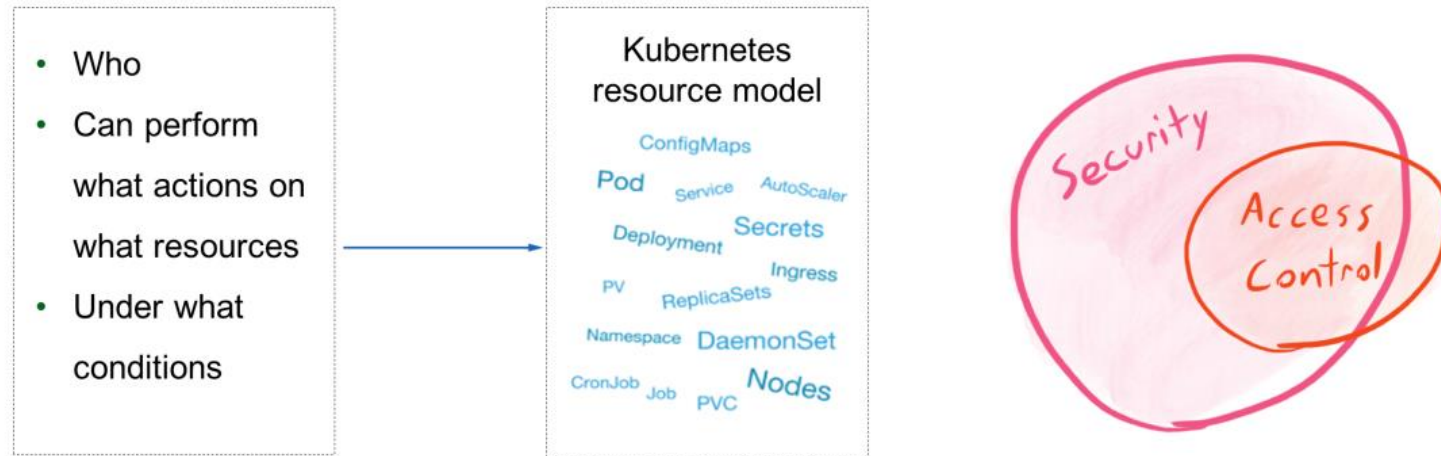
Session 9

Kubernetes (Cont.)

Agenda

- Assignment Review & Guides
- Kubernetes authentication, authorization and admission control
- Ingress and ingress controller

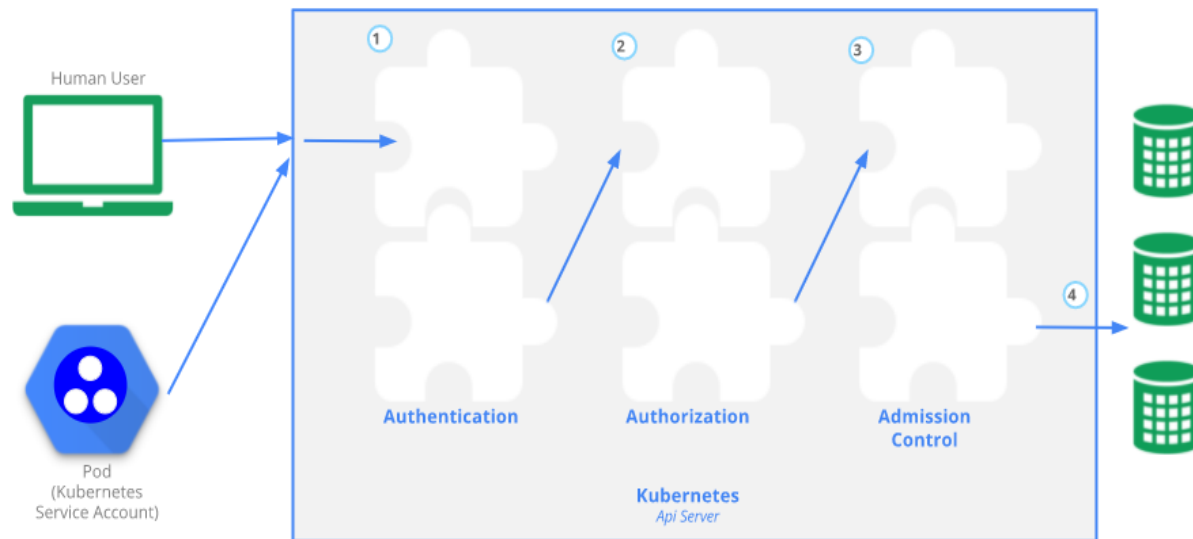
Access control is an important part of cloud-native security. It is also a basic security measure that a Kubernetes cluster must take in a multi-tenant environment



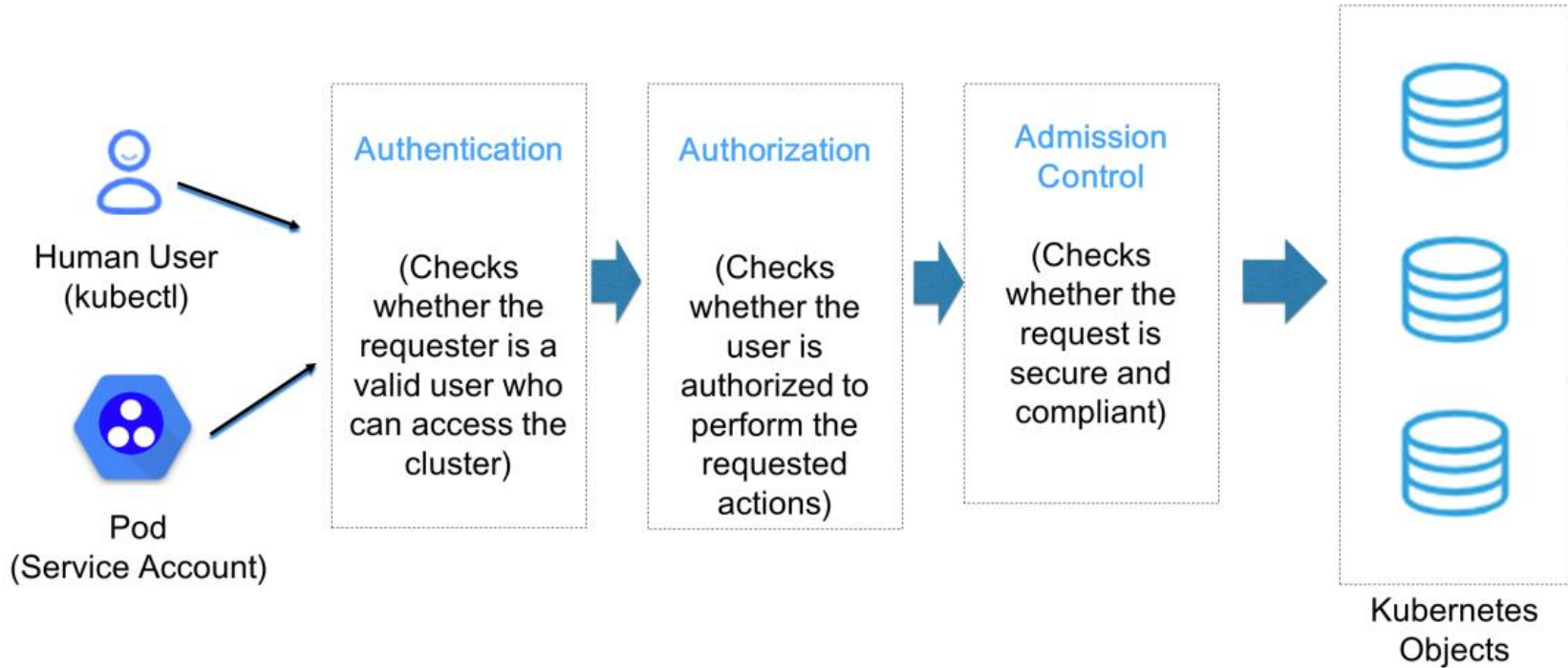
Controlling Access to the Kubernetes API

Users access the Kubernetes API using kubectl, client libraries, or by making REST requests.

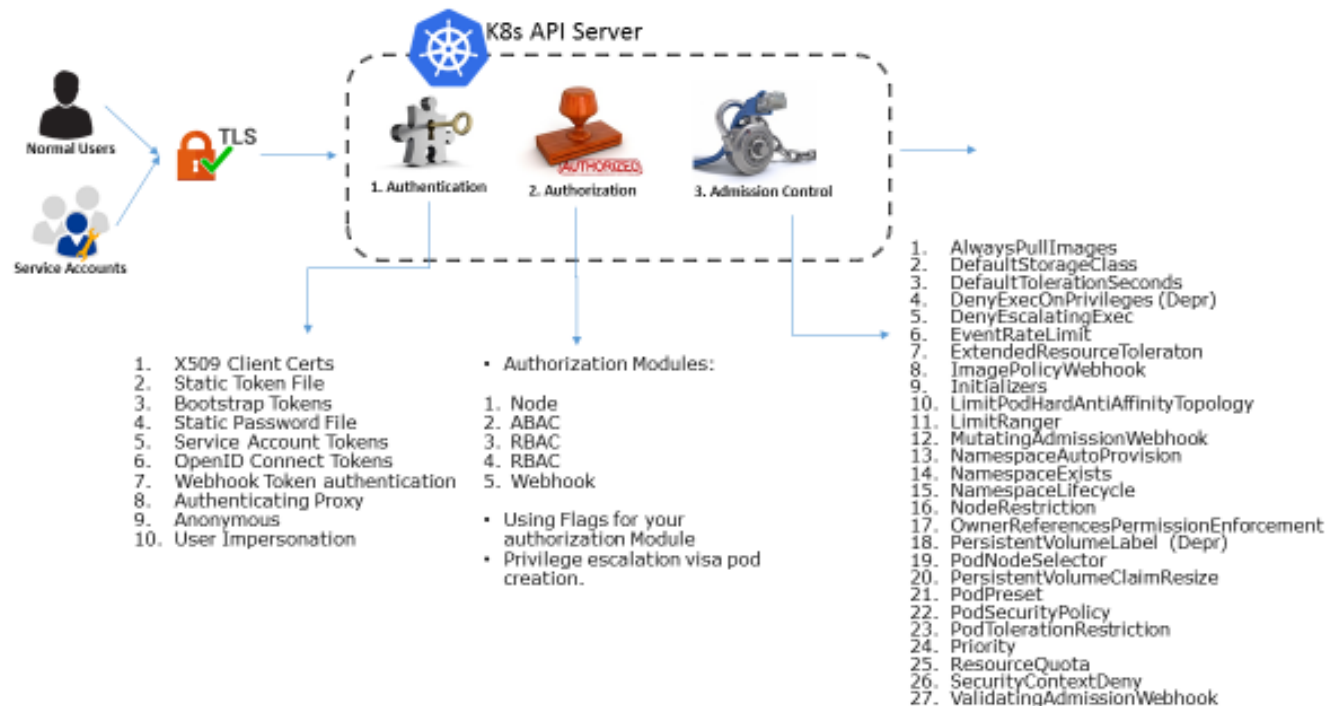
Both human users and Kubernetes service accounts can be authorized for API access



Controlling Access to the Kubernetes API



Accessing the Kubernetes API



Users in Kubernetes

All Kubernetes clusters have two categories of users:

- ❖ Service accounts managed by Kubernetes
- ❖ Normal users

Kubernetes does not have objects which represent normal user accounts. Normal users cannot be added to a cluster through an API call.

Service accounts are users managed by the Kubernetes API. They are bound to specific namespaces, and created automatically by the API server or manually through API calls

Kubernetes uses client certificates, bearer tokens, an authenticating proxy, or HTTP basic auth to authenticate API requests through authentication plugins

You can enable multiple authentication methods at once. You should usually use at least two methods:

- ❖ Service account tokens for service accounts
- ❖ At least one other method for user authentication.

This authentication mode is often used by the API server. The API server starts the Transport Layer Security (TLS)-based handshake process when receiving an access request that is initiated through the client certificate which is signed by the cluster-dedicated Certificate Authority (CA) or by the trusted CA in the API server's client CA.

The API server checks the client certificate validity and verifies the information in the certificate, such as the request source address. X.509 authentication supports two-way authentication and provides robust security.

By default, it is used by Kubernetes components to authenticate each other and provides access credentials that are often used by kube-config for the kubectl client

A service account is an automatically enabled authenticator that uses signed bearer tokens to verify requests

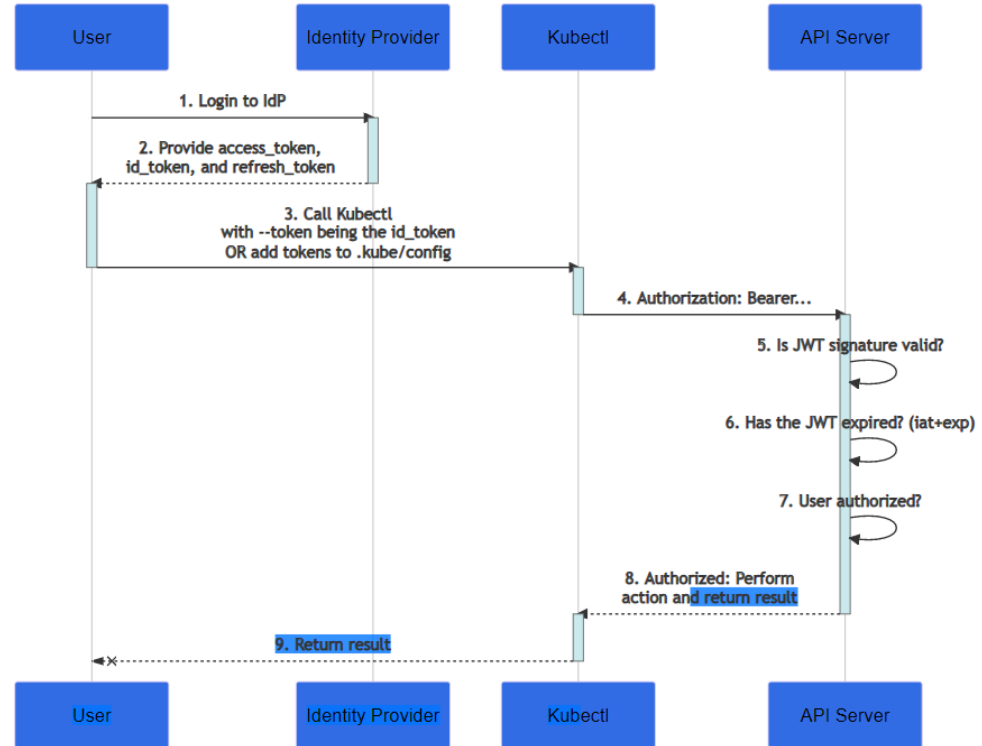
Service accounts are usually created automatically by the API server and associated with pods running in the cluster. Bearer tokens are mounted into pods at well-known locations, and allow in-cluster processes to talk to the API server

```
apiVersion: apps/v1 # this apiVersion is relevant as of Kubernetes 1.9
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: default
spec:
  replicas: 3
  template:
    metadata:
      # ...
    spec:
      serviceAccountName: bob-the-bot
      containers:
      - name: nginx
        image: nginx:1.14.2
```

OpenID Connect Tokens

OpenID Connect is a flavor of OAuth2 supported by some OAuth2 providers, notably Azure Active Directory, Salesforce, and Google

To identify the user, the authenticator uses the `id_token` (not the `access_token`) from the OAuth2 token response as a bearer token



In Kubernetes, you must be authenticated (logged in) before your request can be authorized (granted permission to access)

Kubernetes expects attributes that are common to REST API requests. This means that Kubernetes authorization works with existing organization-wide or cloud-provider-wide access control systems which may handle other APIs besides the Kubernetes API.

Kubernetes authorizes API requests using the API server. It evaluates all of the request attributes against all policies and allows or denies the request. All parts of an API request must be allowed by some policy in order to proceed. This means that permissions are denied by default.

When multiple authorization modules are configured, each is checked in sequence. If any authorizer approves or denies a request, that decision is immediately returned and no other authorizer is consulted. If all modules have no opinion on the request, then the request is denied. A deny returns an HTTP status code 403

The Kubernetes API server may authorize a request using one of several authorization modes:

- ❖ Node
- ❖ ABAC (Attribute-based access control)
- ❖ RBAC (Role-based access control)
- ❖ Webhook

kubectl provides the **auth can-i** subcommand for quickly querying the API authorization layer. The command uses the **SelfSubjectAccessReview** API to determine if the current user can perform a given action, and works regardless of the authorization mode used.

```
kubectl auth can-i create deployments --namespace dev
```

From a practical standpoint, the most useful authorization method is RBAC. It is based on declarative permissions definitions and cluster API objects. The main objects are roles and cluster roles, both representing a set of permissions on certain objects in the API

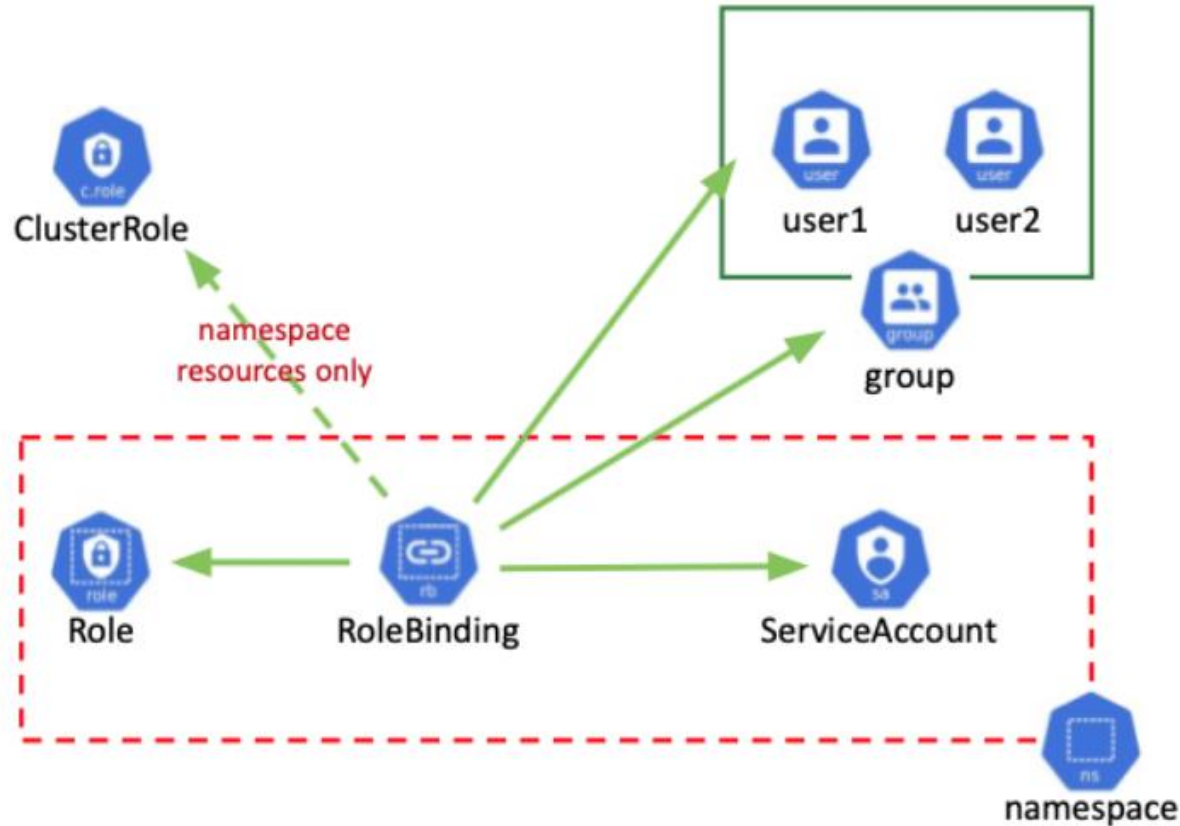
A **role** can only exist in a certain namespace and can only refer to namespaced resources. It also only defines permission to objects within that namespace.

A **cluster role**, on the other hand, is a global object that will provide access to global objects as well as non-resource URLs, such as API version and healthz endpoints on the API server.

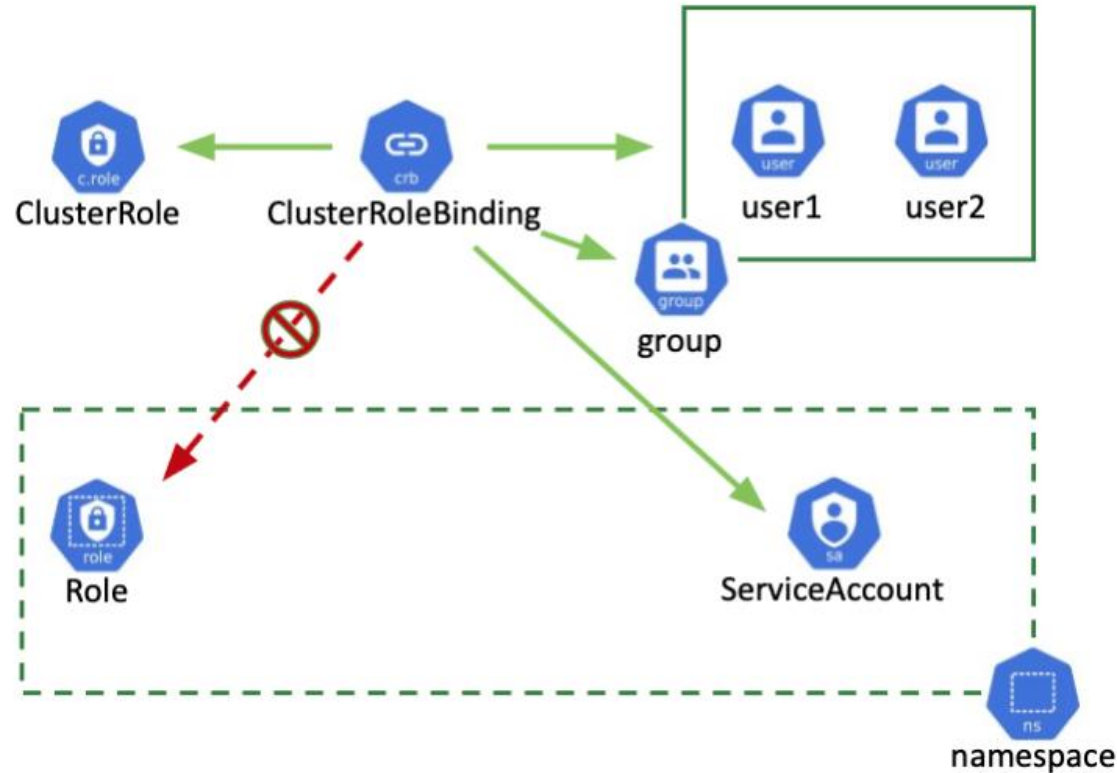
A role binding grants the permissions defined in a role to a user or set of users. It holds a list of subjects (users, groups, or service accounts), and a reference to the role being granted. A RoleBinding grants permissions within a specific namespace whereas a ClusterRoleBinding grants that access cluster-wide.

A **RoleBinding** may reference any Role in the same namespace. Alternatively, a RoleBinding can reference a ClusterRole and bind that ClusterRole to the namespace of the RoleBinding. If you want to bind a ClusterRole to all the namespaces in your cluster, you use a ClusterRoleBinding

RoleBinding and ClusterRoleBinding



RoleBinding and ClusterRoleBinding



Attribute-based access control (ABAC) defines an access control paradigm whereby access rights are granted to users through the use of policies which combine attributes together

To enable ABAC mode, specify `--authorization-policy-file=SOME_FILENAME` and `--authorization-mode=ABAC` on startup.

The file format is one JSON object per line. There should be no enclosing list or map, only one map per line. Each line is a "policy object", where each such object is a map with the properties

```
{"apiVersion": "abac.authorization.kubernetes.io/v1beta1", "kind": "Policy", "spec": {"user": "alice", "namespace": "*", "resource": "*", "apiGroup": "*"}}
```

Node authorization is a special-purpose authorization mode that specifically authorizes API requests made by kubelets

The Node authorizer allows a kubelet to perform API operations including:

- ❖ Read operations
- ❖ Write operations
- ❖ Auth-related operations

A **WebHook** is an HTTP callback: an HTTP POST that occurs when something happens; a simple event-notification via HTTP POST. A web application implementing WebHooks will POST a message to a URL when certain things happen.

When specified, mode Webhook causes Kubernetes to query an outside REST service when determining user privileges

An admission controller is a piece of code that intercepts requests to the Kubernetes API server prior to persistence of the object, but after the request is authenticated and authorized

There are two special controllers: `MutatingAdmissionWebhook` and `ValidatingAdmissionWebhook`. These execute the mutating and validating (respectively) admission control webhooks which are configured in the API

Turn on an admission controller

The Kubernetes API server flag `enable-admission-plugins` takes a comma-delimited list of admission control plugins to invoke prior to modifying objects in the cluster.

For example, the following command line enables the `NamespaceLifecycle` and the `LimitRanger` admission control plugins:

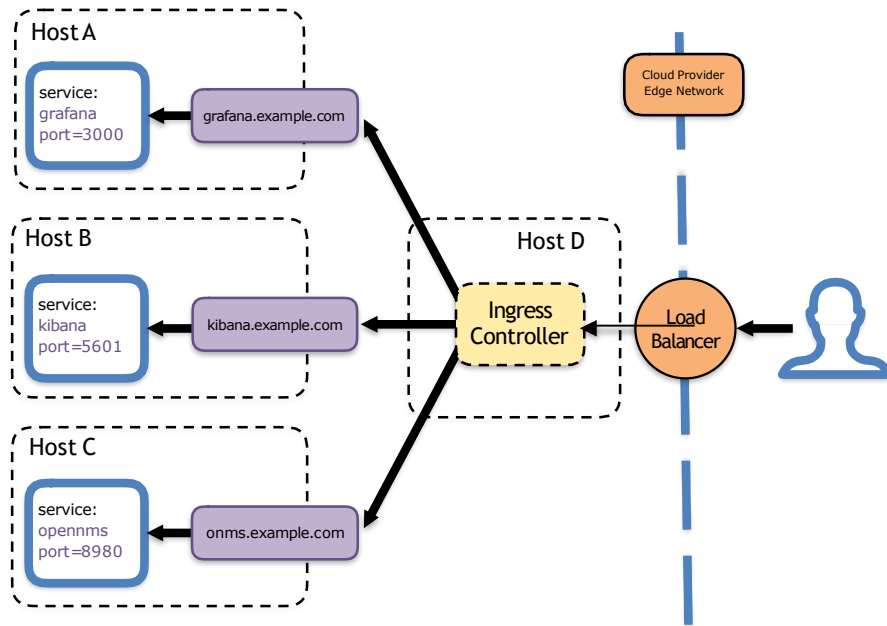
```
kube-apiserver --enable-admission-plugins=NamespaceLifecycle,LimitRanger ...
```

Kubernetes ingress

Known as
Routes in
OpenShift

- An API object that manages external access to the services in a cluster, through a single LoadBalancer.
- Provides load balancing, SSL termination and name/path-based virtual hosting
- Gives services externally-reachable URLs.
- Multiple implementations to choose from.

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-rules
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  rules:
  - host: onms.example.com
    http:
      paths:
      - path: /
        backend:
          serviceName: opennms
          servicePort: 8980
  ...
```



A minimal Ingress resource example:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

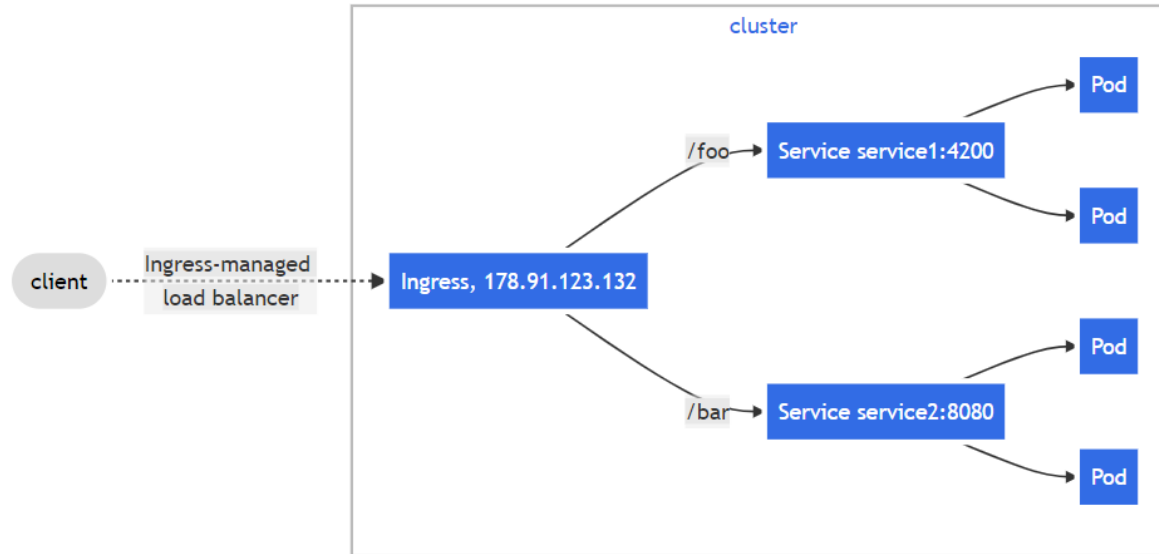
Ingress backed by a single Service

There are existing Kubernetes concepts that allow you to expose a single Service
You can also do this with an Ingress by specifying a default backend with no rules

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: test-ingress
spec:
  defaultBackend:
    service:
      name: test
      port:
        number: 80
```

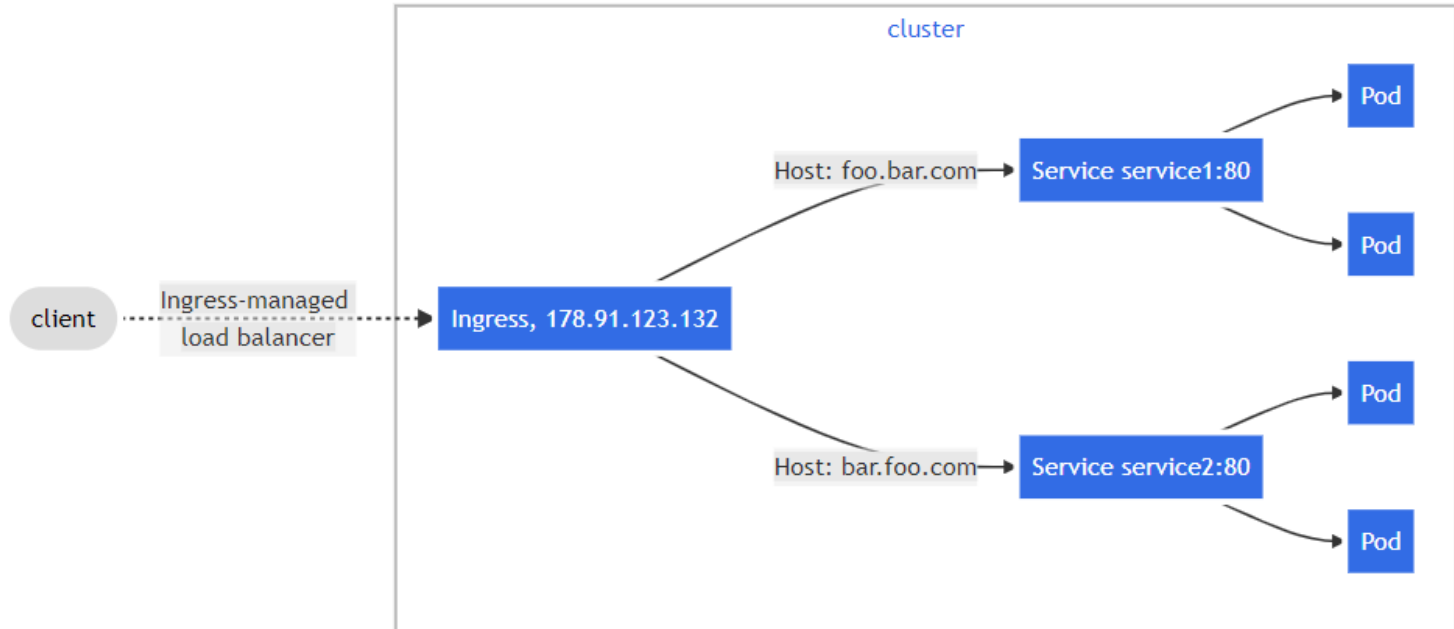
Simple fanout

A fanout configuration routes traffic from a single IP address to more than one Service, based on the HTTP URI being requested. An Ingress allows you to keep the number of load balancers down to a minimum



Name based virtual hosting

Name-based virtual hosts support routing HTTP traffic to multiple host names at the same IP address.



In order for the Ingress resource to work, the cluster must have an ingress controller running.

Unlike other types of controllers which run as part of the kube-controller-manager binary, Ingress controllers are not started automatically with a cluster

Kubernetes as a project supports and maintains AWS, GCE, and nginx ingress controllers

LAB

